



Detección de cambios en interfaces web para procesos RPA utilizando Inteligencia Artificial

Comparación de Modelos

TRAZABILIDAD DEL DOCUMENTO

Versión	Preparado Por	Fecha documento	Revisado por	Descripción
1	Andrés Martín Cantos Rivadeneira María Paola Mendoza Mendieta	24-09-25	PhD. Gladys Villegas	Versión Inicial

CONTENIDO

CARATULA.....	1
TRAZABILIDAD DEL DOCUMENTO	2
CONTENIDO	3
1. DESCRIPCIÓN TEÓRICA.....	4
1.1.- YOLO (You Only Look Once – One-Stage Detector).....	4
1.2.- Faster R-CNN (Two-Stage Detection)	6
1.3.- RetinaNet	7
1.4.- Siamese CNN.....	9
2. VENTAJAS Y DESVENTAJAS	10
3. COMPLEJIDAD COMPUTACIONAL	11
4. CASOS DE USO TÍPICOS	12
5. APLICABILIDAD A SU PROYECTO.....	13
Bibliografía	15

1. DESCRIPCIÓN TEÓRICA

1.1.- YOLO (You Only Look Once – One-Stage Detector)

Fundamentos teóricos:

- YOLO es un modelo y marco unificado de visión artificial desarrollado por Ultralytics.
- Está especializado en la detección de objetos en tiempo real.
- Se basa en redes neuronales convolucionales (CNNs).
- A diferencia de otros métodos, procesa la imagen completa en una sola pasada.
- Predice de manera simultánea las cajas delimitadoras (bounding boxes) y las clases de los objetos

Fundamentos matemáticos ¿Cómo funciona?

- Divide la imagen en una grilla $S \times S$.
- Cada celda predice:

$$(x, y, w, h, C, p_1, p_2, \dots, p_k)$$

donde:

- (x, y) : centro del objeto relativo a la celda.
- (w, h) : ancho y alto relativo a la imagen.
- C : confianza = $P(\text{objeto}) \times IoU$.
- p_i : probabilidad de clase i .
- Resultado final de detección:

$$\text{Score}(i) = C \times p_i$$

En esencia: YOLO = optimización conjunta de cajas + confianza + clasificación en una sola pasada de red convolucional [1].

Usaremos la versión YOLOv8 lanzada el 10 enero 2023 y es la última estable.

Hay algunas variantes:

Modelo	Tamaño / parámetros	Velocidad y precisión	Uso típico
Nano(n)	~1.1M	Muy rápido – precisión baja o moderada	Prototipos, pruebas rápidas y baja VRAM
Small(s)	~7.5M	Rápido – Precisión Buena	Bots que necesitan equilibrio velocidad/precisión

Medium(m)	~21M	Moderado – Precisión Mejor	Objetos pequeños o más complejos, UI con muchos elementos
Large(l)	~47M	Más lento – Precisión Alta	Alta precisión, objetos pequeños, se necesita GPU potente
Xlarge(x)	~87M	Muy Lento – Precisión Muy alta	Máxima precisión, mucha VRAM, entrenamiento más largo

Usaremos la versión Nano YOLOV8n

Tipo de Aprendizaje: Supervisado

- En el entrenamiento se usan imágenes junto con sus anotaciones.
- Las anotaciones se leen automáticamente desde archivos .txt.
- Para cada imagen (imagen.png) debe existir un archivo de etiquetas (imagen.txt).
- Los archivos de etiquetas deben estar en una carpeta relativa llamada /labels.

Parámetros Principales:

Parámetro	Descripción
data	Se configura con un archivo .yaml donde se define la ruta del dataset de imágenes y las clases
Epochs	Cantidad de veces que el modelo ve todo el dataset
Imgsz	Tamaño a la que escala la imagen, a mayor tamaño requiere más RAM. Si se quiere precisión en objetos pequeños requiere imágenes más grandes
Batch	Cantas imágenes se procesan en cada paso del entrenamiento, estabilidad del entrenamiento, más grande es más estabilidad del entrenamiento, mejor estimación del gradiente, pero mayor consumo de procesamiento y memoria
Workers	Número de hilos para cargar los datos.

1.2.- Faster R-CNN (Two-Stage Detection)

Fundamentos teóricos:

- Faster R-CNN es una arquitectura de red neuronal para detección de objetos.
- Integra la generación de propuestas de región dentro de la red, permitiendo un entrenamiento de extremo a extremo.
- Es un modelo de dos etapas:
 - RPN (Region Proposal Network): propone regiones candidatas.
 - Segundo módulo: clasifica y ajusta las cajas.
- Es muy preciso, incluso con objetos pequeños o superpuestos.
- Requiere más cómputo que otros métodos más rápidos.
- PyTorch (torchvision) ofrece implementaciones pre-entrenadas de Faster R-CNN.

Fundamentos matemáticos ¿Cómo funciona?

- **Etap 1 – RPN (Region Proposal Network):**
Predice un conjunto de regiones $R = \{r_1, r_2, \dots, r_n\}$ con:
$$(x, y, w, h, C)$$

donde (x, y) es el centro, (w, h) tamaño, y $C = P(\text{objeto})$.
- **Etap 2 – Clasificación y ajuste:**
Para cada región propuesta r_i , la red predice:
$$(p_1, p_2, \dots, p_k, \Delta x, \Delta y, \Delta w, \Delta h)$$
 - p_j : probabilidad de clase j .
 - Δ : ajustes para refinar la caja propuesta.
- **Resultado final:**
$$\text{Score}(i, j) = C_i \times p_j$$

con cajas ajustadas:
$$B' = B + \Delta$$

En esencia: propone regiones + clasifica + ajusta cajas → detección precisa [2].

Tipo de Aprendizaje: Supervisado

- Se necesita un conjunto de datos etiquetados con:
 - Imágenes.
 - Coordenadas de bounding boxes.
 - Clases de los objetos en cada imagen.
- El dataset que usaremos está en formato YOLO (.txt), por lo que es necesario convertirlo a formato JSON compatible con PyTorch.

Parámetros Principales:

Parámetro	Descripción
-----------	-------------

Tasa de aprendizaje (learning rate)	Determina el tamaño de los "pasos" que da el optimizador para actualizar los pesos del modelo. Una tasa demasiado alta puede hacer que el entrenamiento sea inestable, mientras que una muy baja puede hacerlo demasiado lento.
Epochs	Cantidad de veces que el modelo ve todo el dataset
Optimizador	El algoritmo que se utiliza para minimizar la función de pérdida y actualizar los pesos del modelo. El más común es el descenso de gradiente estocástico (SGD), a menudo con momentum = 0.9 para acelerar la convergencia.
Batch	Cantas imágenes se procesan en cada paso del entrenamiento, estabilidad del entrenamiento, más grande es más estabilidad del entrenamiento, mejor estimación del gradiente, pero mayor consumo de procesamiento y memoria
Regularización (Weight decay)	Una técnica para prevenir el sobreajuste al añadir un pequeño valor a la función de pérdida, lo que anima al modelo a mantener sus pesos pequeños.

1.3.- RetinaNet

- RetinaNet es un modelo de detección de objetos de una sola etapa (one-stage detector).
- Su aporte principal es la Función de Pérdida Focal (Focal Loss).
- La Focal Loss resuelve el desequilibrio de clases: muchas regiones son fondo y pocas contienen objetos.
- Este desequilibrio dificulta el entrenamiento de detectores de una sola etapa.
- Matemáticamente, la Focal Loss modifica la entropía cruzada (cross-entropy) con un factor que:
 - Reduce el peso de ejemplos fáciles (ya bien clasificados).
 - Enfatiza ejemplos difíciles, mejorando el aprendizaje.

Fundamentos matemáticos ¿Cómo funciona?

- Para cada ancla predice:

$$(x, y, w, h, p_1, p_2, \dots, p_k)$$

donde (x, y, w, h) son coordenadas de la caja y p_j la probabilidad de clase j .

- Usa la **Focal Loss** para la clasificación:

$$FL(p_t) = -\alpha(1 - p_t)^\gamma \log(p_t)$$

- p_t : probabilidad predicha para la clase correcta.
- α : factor de balanceo.
- γ : factor de modulación que reduce el peso de ejemplos fáciles.

En resumen, detecta objetos en una sola etapa corrigiendo el desequilibrio entre fondo y objetos [3].

Tipo de Aprendizaje: Supervisado

- Requiere un conjunto de datos etiquetado con:
- Imágenes.
 - Coordenadas de bounding boxes.
 - Clases de los objetos.
- El modelo aprende a predecir estas anotaciones a partir de los datos de entrada.
- El dataset que usaremos está en formato YOLO (.txt), por lo que es necesario convertirlo a formato JSON compatible con RetinaNet.

Parámetros Principales:

Parámetro	Impacto
Hiperparámetro Gamma (γ) de Focal Loss	Controla qué tan fuerte es la modulación de la pérdida para los ejemplos fáciles. Valores típicos están entre 0.5 y 5.
Tasa de aprendizaje (Learning rate)	Como en cualquier red neuronal, determina el tamaño de los pasos de actualización de los pesos.
Número de clases (Number of classes)	La cantidad de categorías de objetos que el modelo debe detectar.
Parámetros de la red backbone	Dependiendo de la arquitectura subyacente (por ejemplo, ResNet-50, ResNet-101), estos parámetros definen la capacidad y la complejidad del modelo.

1.4.- Siamese CNN

- Una Siamese CNN tiene dos o más ramas idénticas de CNN que comparten pesos.
- La salida no es una clasificación directa, sino un vector de características (embedding).
- Se compara la similitud entre dos entradas usando distancia euclidiana o distancia de coseno.
- Se enfoca en comparar imágenes para detectar similitudes o diferencias, no en localizar objetos.
- Aprende una función de distancia $G(x)$ que transforma una imagen en un vector de características.
- La red se entrena para que la distancia entre embeddings sea pequeña si las imágenes son similares y grande si son diferentes.
- La función de pérdida más común es la Contrastive Loss (Pérdida de Contraste).

Fundamentos matemáticos ¿Cómo funciona?

- Cada entrada x_1, x_2 pasa por la misma red $G(\cdot)$ para generar embeddings:

$$v_1 = G(x_1), \quad v_2 = G(x_2)$$

- Se calcula una **distancia** entre los embeddings, por ejemplo:

- **Euclidiana:** $D = \|v_1 - v_2\|_2$

- **Coseno:** $D = 1 - \frac{v_1 \cdot v_2}{\|v_1\| \|v_2\|}$

- La red se entrena con **Contrastive Loss**:

$$L = (1 - y) \frac{1}{2} D^2 + y \frac{1}{2} \max(0, m - D)^2$$

donde:

- $y = 0$ si las imágenes son similares, $y = 1$ si son diferentes.
- m es un margen que separa los embeddings de imágenes diferentes.

Básicamente aprende un mapeo $G(x)$ que coloca imágenes similares cerca y diferentes lejos en el espacio de embeddings [4].

Tipo de Aprendizaje: Supervisado

- Requieren pares de imágenes etiquetados para entrenar:
 - Similares (pares positivos)
 - Diferentes (pares negativos)
- El entrenamiento se basa en aprender a distinguir similitudes y diferencias entre las imágenes.
- El dataset que usaremos está en formato YOLO (.txt), por lo que es necesario:

- Usar las imágenes y alterarlas levemente para los pares positivos.
- Usar imágenes diferentes para los pares negativos.

Parámetros Principales:

Parámetro	Descripción
Arquitectura de la red base (backbone)	La red convolucional subyacente que extrae las características. Por ejemplo, se pueden usar redes como VGG16 o ResNet.
Función de Pérdida	La elección de la función de pérdida es crítica. La Pérdida de Contraste y la Pérdida de Triplete (Triplet Loss) son las más comunes. La pérdida de triplete compara tres imágenes a la vez: un ancla, una positiva y una negativa.
Distancia métrica	La función utilizada para medir la similitud entre los vectores de salida. La distancia euclidiana y la distancia del coseno son las más comunes
Margen	Un hiperparámetro de la función de pérdida (como la pérdida de contraste) que define el umbral de separación deseado entre los pares positivos y negativos.

2. VENTAJAS Y DESVENTAJAS

Modelo	Ventajas	Desventajas
YOLO	Muy rápido, adecuado para detección en tiempo real. Procesa toda la imagen en una sola pasada (one-stage detector). Simplicidad de implementación.	Menos preciso con objetos pequeños o muy cercanos. Sensible a cambios de escala y superposición.
Faster R-CNN	Muy preciso, especialmente con objetos pequeños o superpuestos. Entrenamiento de extremo a extremo. Buen rendimiento en datasets complejos.	Computacionalmente intensivo, no ideal para tiempo real. Requiere más memoria y recursos.

RetinaNet	Combina rapidez de un detector de una sola etapa con buena precisión. La Focal Loss maneja desequilibrio entre fondo y objetos. Adecuado para escenarios con muchas clases y objetos pequeños.	Más complejo que YOLO en implementación. Requiere ajuste cuidadoso de hiperparámetros de la Focal Loss. Aún más costoso que YOLO en cómputo. El dataset del que disponemos no se adapta muy bien a este modelo
Siamese CNN	Permite comparar similitud entre imágenes. Útil para detección de cambios, verificación de identidad o comparación de patrones. Aprende un espacio de embeddings donde las imágenes similares están cerca.	No predice ubicación de objetos, solo similitud. Requiere pares de entrenamiento cuidadosamente etiquetados. Menos aplicable a detección directa de múltiples objetos.

3. COMPLEJIDAD COMPUTACIONAL

Modelo	Complejidad temporal de entrenamiento	Complejidad temporal de predicción	Complejidad espacial	Escalabilidad con el tamaño de datos
YOLO	$O(N)$, donde N = número de imágenes $\times$ tamaño de la red Rápido por procesamiento en una sola pasada	$O(1)$ por imagen (muy rápido)	Moderada: depende del tamaño de la red	Alta, escala bien a datasets grandes
Faster R-CNN	$O(N \cdot R)$ R = número de propuestas de región	$O(R)$ por imagen más lento que YOLO	Alta: necesita memoria para RPN y features	Media, puede ser costoso con muchos datos o imágenes grandes

RetinaNet	$O(N)$, similar a YOLO, pero mayor por la Focal Loss y múltiples anclas	$O(A)$ por imagen A = número de anclas, rápido, pero menos que YOLO	Moderada-alta: depende del número de anclas	Alta, escala bien pero más costoso que YOLO
Siamese CNN	$O(N \cdot P)$, P = número de pares de entrenamiento	$O(1)$ por par de imágenes	Moderada: depende del tamaño de embeddings	Media, crecimiento de pares es cuadrático si se generan todos los pares posibles

4. CASOS DE USO TÍPICOS

Modelo	Dominios de aplicación	Tipos de datos apropiados	Ejemplos reales de implementación	Industrias que lo utilizan
YOLO	Detección de objetos en tiempo real	Imágenes y video	Detección de peatones en cámaras de seguridad, seguimiento de vehículos	Seguridad, transporte, robótica, retail
Faster R-CNN	Detección precisa de objetos y análisis detallado	Imágenes con objetos pequeños o superpuestos	Reconocimiento de células en microscopía, detección de defectos en manufactura	Medicina, manufactura, investigación científica
RetinaNet	Detección de objetos en escenarios con muchas clases o desequilibrio de fondo	Imágenes con múltiples objetos y clases	Detección de señales de tráfico, conteo de productos en almacenes	Transporte, logística, retail, tráfico inteligente
Siamese	Comparación de similitud,	Pares de imágenes	Reconocimiento facial,	Seguridad, banca, QA de

CNN	verificación, detección de cambios		verificación de firmas, comparación de interfaces web	software, análisis forense
------------	------------------------------------	--	---	----------------------------

5. APLICABILIDAD A SU PROYECTO

Modelo	Relevancia específica al proyecto	Viabilidad de implementación	Recursos necesarios	Justificación de inclusión/exclusión
YOLO	Alta: para detectar objetos en interfaces no compara cambios entre dos estados de la interfaz	Alta: fácil de implementar con librerías existentes	GPU moderada + 12 GBRAM, dataset etiquetado Es necesario reducir el tamaño de las imágenes del dataset, pero no tanto. Se puede entrenar con los recursos gratuitos del Google Colab	Incluido: Tiene un costo computacional aceptable. Se puede utilizar para detectar objetos en las interfaces web, en la segunda etapa de la arquitectura de la solución, para poder detectar cambios menores y modificar los parámetros del RPA
Faster R-CNN	Media: preciso en detección de objetos no compara cambios entre dos estados de la interfaz	Media-baja: más complejo de entrenar y más lento	GPU potente + 12 GBRAM, y dataset etiquetado. Es necesario reducir bastante el tamaño de las imágenes que tenemos para que pueda realizar el entrenamiento,	Excluido: alto costo computacional, se podría utilizar, pero habría que bajar mucho la resolución de las imágenes para usar los recursos gratuitos de colab y se perdería detalle.

			caso contrario falla por falta de recursos en la versión gratuita del Google Colab	
RetinaNet	Media: buena precisión con múltiples objetos no compara cambios entre dos estados de la interfaz	Media-baja: implementación posible, pero es complejo realizar ajuste de Focal Loss Además, el dataset del que disponemos no se adapta bien para este CNN	GPU + 12GB RAM, y dataset complejo con imágenes y anotaciones especiales en formato json	Excluido: alto costo computacional y requiere un dataset etiquetado más complejo
Siamese CNN	Alta: diseñada para comparar similitud entre dos imágenes, ideal para detectar cambios	Alta: relativamente sencillo con dataset de pares positivos/negativos	GPU + 12GB RAM y dataset etiquetado. GPU para más velocidad y comodidad, pero se logra realizar el entrenamiento de 200 imágenes con CPU	Incluido: directamente aborda la detección de cambios entre interfaces, enfoque adecuado para RPA Se puede incluir en las primeras etapas de la arquitectura de la solución para saber a primera vista si la interfaz tiene cambios significativos o no.

Bibliografía

- [1] R. Joseph, D. Santosh, G. Ross y F. Ali, «You Only Look Once: Unified, Real-Time Object Detection,» *arXiv (Cornell University)*, 1 Enero 2015.
- [2] S. Ren, K. He, R. Girshick y J. Sun, «Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks,» *arXiv (Cornell University)*, Enero 2015.
- [3] T.-Y. Lin, P. Goyal, R. Girshick, K. He y P. Dollár, «Focal loss for dense object detection,» *arXiv (Cornell University)*, 2017.
- [4] G. Koch, R. Zemel y R. Salakhutdinov, «Siamese Neural Networks for One-shot Image Recognition,» *Proceedings of the 32nd International Conference on Machine Learning, Lille, Francia*, vol. 37, 2015.